

Final Project
Natural Language Processing
Master in Machine Learning for Health
Academic year 2025/2026

March 25, 2026

Contents

1	Introduction	3
2	Project Guidelines	3
2.1	Functional specifications	3
2.2	Working Groups	4
2.3	Submission	4
3	Peer Review	5
4	Evaluation	6
5	Description of the Practical Applications	7
5.1	Chatbot for Answering Questions from a Document Database	7
5.2	Fact-Checking System	8
5.3	Academic Research Assistance System	9
6	Deployment Environment at UC3M	10

1 Introduction

Large language models (LLMs) have revolutionized artificial intelligence, as well as the way we approach many tasks in our professional and everyday lives. Their true value, however, lies in their ability to tackle real-world problems beyond conventional uses, enabling advanced solutions in areas such as customer service, fact-checking, and research assistance.

Throughout this course, we have seen how the invention of the Transformer architecture in 2017 marked a turning point in natural language processing (NLP). By introducing attention mechanisms to handle long-range dependencies, Transformers eliminated the need for sequential models such as recurrent neural networks. The original **Encoder-Decoder** Transformer architecture is particularly effective for sequence transformation tasks like machine translation and text summarization. Additionally, two specialized variants exist: **Encoder-based** models (e.g., BERT), which excel at text analysis tasks such as classification and question answering, and **Decoder-based** models (e.g., GPT), which are designed for text generation.

The shift toward large language models has been primarily driven by the expansion and adaptation of these Decoder-based models. By dramatically increasing the number of parameters and the volume of training data, these models have developed a greater capacity to capture complex linguistic patterns, general world knowledge, and diverse linguistic styles.

However, LLMs are not infallible; they can generate inaccurate or fabricated responses—a phenomenon known as “*hallucination*”. This is where **Retrieval-Augmented Generation (RAG)** systems come into play. RAG aims to mitigate hallucinations and improve response accuracy by integrating additional sources of knowledge. These systems are particularly valuable when an LLM needs to provide precise answers based on a specific set of documents rather than relying solely on the general knowledge acquired during pre-training.

Objective

The objective of this project is to help students develop a practical understanding of iterative workflows with language models and RAG systems through the creation of three applications:

- A. A chatbot that allows answering questions based on a document database
- B. A fact-checking system
- C. An academic research assistance system

Project Structure

The project consists of two mandatory and interconnected components:

1. **System Implementation** (Section 2): Each group develops one of the three applications listed above.
2. **Peer Review** (Section 3): Each group member individually reviews submissions from groups working on the other two application types, following ACL review criteria.

Both components are mandatory and contribute to the final grade.

2 Project Guidelines

2.1 Functional specifications

Each application must meet a set of **mandatory functional requirements**:

- Students **must use a vector database** to implement the RAG system. Direct queries on sources such as Wikipedia are **not valid**.
- All systems must be **evaluated based on the specified criteria** (Table 1) and all parameters used for the deployment of the system (e.g., temperature, top-k in the retrieval step of the RAG, etc.) must be validated and justified.
- **For the basic project, only open-source LLMs can be used.**

- In all cases, a basic frontend must be developed using tools such as Streamlit¹ to enhance the presentation.

Students may enhance their system with **additional features**, either by incorporating suggested ideas or proposing their own. While these features are optional, **choosing not to implement any will cap the maximum grade at 8.**

2.2 Working Groups

Students will work in teams of four or three members (**6 groups of 4 members and 1 group of 3**). Each group must select one of the three designated applications by **March 27. Group selection will be managed through Aula Global.**

2.3 Submission

Students must submit the following:

1. A GitHub repository with the implemented code, properly commented.
2. A **demo video** of their system of **maximum 2 minutes**.
3. A **paper-style report** of **maximum 4 pages**, following academic paper conventions that must include:
 - The link to the GitHub repository and the video (students can either upload it to Drive or YouTube).
 - A concise description of the design decisions, including but not limited to dataset selection, technologies used, database configuration, and any additional features implemented.
 - Configuration details for both the LLM and the vector database, as well as optimization techniques (parameter tuning, prompting strategies, etc.).
 - An explanation of how the system meets the functional requirements, how hallucinations are mitigated, and any other detail that you consider makes your work noteworthy / innovative.
 - Evaluation of the system based on the criteria in Table 1. Vague statements such as “*the system works because it provides answers formatted correctly or appears to respond appropriately*” are insufficient—**specific metrics must be provided to validate performance.**
 - Conclusions and limitations of the system. Omitting either of these sections will result in a penalty. These sections must be specific and directly tied to the work performed—generic statements such as “*one future direction is to reduce the system’s hallucinations*” are insufficient. Instead, the report should analyze why hallucinations occur, under what circumstances, and provide concrete suggestions for improvement.

The report must *not* include explanations of standard methods—it should focus on design decisions, experimental results, and conclusions.

4. A presentation session that will take place on **May 6**.

Criterion	Chatbot	Fact-Checking System	Research Assistance
<i>RAG System Quality</i>	Retrieves relevant information and generates coherent, useful responses.	Retrieves relevant evidence and provides precise / accurate verdicts on claim veracity	Retrieves key articles and accurately compares them with prior research.
<i>Document Coverage</i>	Percentage of queries answered using database information.	Ability to find sufficient evidence to verify claims.	Ability to retrieve relevant articles related to the research topic.
<i>Response Time</i>	Speed at which responses are generated.	Speed at which claims are verified and verdicts provided.	Speed of article retrieval and comparison generation.

Table 1: Evaluation criteria for each application.

The PDF will be the only document uploaded to Aula Global, and only one group member needs to submit it. The deadline is April 30 at 23:59.

¹<https://github.com/streamlit/streamlit>

3 Peer Review

Each student will be randomly assigned to review a submission of other groups. **Peer review evaluations will be carried out in class on May 5.**

Reviews must follow the following template:

1. **Project Summary:** Describe what the project is about, including the chosen application type and the direction taken by the group, the dataset used, and the main design decisions. Highlight the key contributions and flag any aspects that may have been misunderstood or are unclear.
2. **Summary of Strengths:** What are the major strengths of this work? These could include a well-designed RAG pipeline, thoughtful dataset selection, solid evaluation methodology, creative additional features, or clarity of the demo and report.
3. **Summary of Weaknesses:** What are the main concerns? These could include insufficient evaluation, unjustified parameter choices, hallucination issues that are not addressed, a poorly functioning demo, or lack of clarity in the report.
4. **Comments, Suggestions, and Typos:** Any additional comments to help the authors improve their system or report.
5. **Soundness:** Is the system technically sound and well-evaluated? Consider whether the RAG pipeline is correctly implemented, whether the evaluation criteria from Table 1 are addressed with specific metrics, and whether design decisions are justified.
 - **5 = Excellent:** The system is thoroughly implemented and evaluated. All design decisions are clearly justified.
 - **4.5**
 - **4 = Strong:** The system is well-implemented and evaluated. Some minor aspects could use further justification or testing.
 - **3.5**
 - **3 = Acceptable:** The system works and is reasonably evaluated, but some important decisions lack justification or some evaluation aspects are missing.
 - **2.5**
 - **2 = Poor:** There are notable gaps in the implementation or evaluation. Key design choices are unjustified or the system does not fully meet the functional requirements.
 - **1.5**
 - **1 = Major Issues:** The system is incomplete, non-functional, or does not meet the basic requirements of the assignment.
6. **Excitement:** How impressive or inspiring do you find this project? This is *subjective*. You may find a project exciting because it tackles the problem in a creative way, achieves surprisingly good results, includes a well-designed demo, or implements particularly thoughtful additional features.
 - **5 = Highly Exciting:** This project genuinely impressed me. I would highlight it as an example of excellent work.
 - **4.5**
 - **4 = Exciting:** This project stands out in some meaningful way and goes beyond the basic requirements.
 - **3.5**
 - **3 = Interesting:** This project is solid and has some noteworthy aspects, but does not particularly stand out.
 - **2.5**
 - **2 = Potentially Interesting:** The project is functional but does not go beyond the minimum requirements in any notable way.
 - **1.5**
 - **1 = Not Exciting:** The project does not meet expectations or fails to demonstrate meaningful results.

4 Evaluation

The final grade combines three components:

- **System Implementation** (6 points)

$$SI(6 \text{ points}) = PRES + REQ \cdot (Met + RepQ + CodQ)$$

where

- *REQ*: Compliance with functional requirements ([0,1])
- *Met*: Methodology (3)
- *RepQ*: Quality of the report (0.75)
- *CodQ*: Quality of the code (0.25)
- *PRES*: Presentation (2)

- **Extension Work** (maximum 2 points)

$$EW(2 \text{ points}) = ORIG + SC \cdot Met$$

where

- *ORIG*: Originality (0.5)
- *SC*: Scope. Difficulty and breadth of the additional features implemented ([0,1])
- *Met*: Methodology (1.5)

- **Peer Review** (2 points)

- **As evaluator (1 point)**: Each group member will be assessed on the quality of the reviews they submit, based on the peer-review criteria specified above. This evaluation item will also take into account the coherence between the identified strengths and concerns and the provided soundness and excitement scores.
- **As evaluated (1 point)**: The peer-review scores received from other groups will be taken into account as part of the assessment of the demo video and paper-style report. The final grade for this component will be computed as the average of the Soundness and Excitement scores across the three individual reviews received. If necessary, the teachers will act as meta-reviewers in case of negligent or low-quality reviewers.

5 Description of the Practical Applications

5.1 Chatbot for Answering Questions from a Document Database

A chatbot without RAG may offer imprecise answers due to its lack of access to the relevant documents, relying solely on pre-trained general information. In this exercise, we will focus on overcoming this limitation by building a chatbot with RAG.

Functional Requirements. The implemented chatbot must meet the following requirements:

- The answers must include the specific document or documents that support the response.
- If the database does not contain a relevant document, the chatbot must not provide an answer.
- The answers must be in the same language as the question. This means that responses should match the language in which the user asks the question, which should also correspond to the language of your dataset.

Below is an example of the expected interaction with the chatbot:

User:
What are the requirements for renewing the national ID (DNI) in Spain?

Chatbot:
According to the BOE document with reference BOE-A-2005-21163, the renewal must be carried out in person by the DNI holder. You can find more information [here](#).

User:
How can one access study scholarships?

Chatbot:
I'm sorry, I don't have any information on that in the available BOE documents. Please check the BOE directly or provide more details for a more accurate search.

Database. As a document database, any collection of documents (in plain text or PDF) on which a user might be interested in asking questions can be used. Below are some examples, but students may choose other document sets that they find more interesting or relevant.

- Documents from the Official State Gazette (BOE)². Here³ you can access a collection of 216,000 documents in HTML or XML format.
- Documents from the curriculum of the Master's in Machine Learning for Health (or from other master's programs at UC3M or another university).
- User manuals and online documentation of electronic products, such as the Sony manuals database⁴. Here you can access user manuals and product guides, such as for cameras or televisions, in PDF format.

Additional Features.

- Display suggestions for similar topics. For this, a topic model can be trained on the database, and topic/question suggestions can be shown once the user asks a question.
- Add the option to generate automatic summaries from which information has been extracted, so that the user can obtain a condensed version of the retrieved content. For this, a Transformer model fine-tuned on the training corpus and/or an LLM can be used.
- Allow the chatbot to handle queries in multiple languages, translating both the questions and the answers if necessary, using pre-trained (or fine-tuned) translation models.

²<https://www.boe.es/>

³https://github.com/PlanTL-INTELCOMP/DBimport/tree/master/data.Law_BOE

⁴<https://www.sony.com/electronics/support>

5.2 Fact-Checking System

This application focuses on developing a RAG system that automatically verifies the veracity of a statement using a specific corpus. The system will retrieve relevant information from the corpus and employ an LLM to assess the truthfulness of the statement.

Functional Requirements. The system must meet the following requirements:

- Provide a clear response that explains the veracity of the statement.
- Accompany the responses with the sources and excerpts from the corpus that justify the verdict.
- If the system cannot verify the veracity of the statement due to lack of information, it must indicate so without fabricating an answer.
- The answers must be in the same language as the question. This means that responses should match the language in which the user asks the question, which should also correspond to the language of your dataset.

Below is an example of the expected interaction with the system:

User:

Climate change does not impact the frequency of extreme weather events.

System:

False. According to recent scientific studies, climate change is increasing the frequency of extreme weather events such as hurricanes, heat waves, and floods.

- Increase in temperatures: climate change is causing a rise in global temperatures, which leads to more frequent and intense heat waves [...] [\[source\]](#)
- Increase in temperatures: climate change is causing a rise in global temperatures, which leads to more frequent and intense heat waves [...] [\[source\]](#)

User:

5G technology has a direct impact on human health.

System:

I'm sorry, I do not have enough information in the corpus to verify this claim.

Database. As a source of information to corroborate the facts, any textual database containing reliable information can be used. Below are some examples, but students may choose other document sets that they find more interesting or relevant.

- A Wikipedia dump, which can be easily obtained through the `wikipedia-api` library⁵.
- PubMed. A collection of scientific articles on topics related to medicine or health sciences, available through its website⁶.
- COVID-19. A corpus of academic articles on COVID-19 and research related to the coronavirus, which can be easily downloaded from the following repository⁷.

Additional Features.

- Include a confidence score indicating how certain the model is about the veracity of the statement, based on the evidence retrieved from the corpus.
- Compare performance with a traditional fact-checking system.
- Add the option to generate automatic summaries from which the information has been extracted, so that the user can obtain a condensed version of the retrieved content. For this, a Transformer model fine-tuned on the training corpus and/or an LLM can be used.
- Allow the system to handle queries in multiple languages, translating both the questions and the answers if necessary, using pre-trained (or fine-tuned) translation models.

⁵<https://pypi.org/project/Wikipedia-API/>

⁶<https://ftp.ncbi.nlm.nih.gov/pubmed/baseline/>

⁷<https://github.com/allenai/cord19>

5.3 Academic Research Assistance System

A key aspect of research is comparing current work with previous studies through a state-of-the-art review. This application aims to support researchers by allowing them to specify a research direction, after which the system will retrieve relevant articles on the topic. Additionally, it will highlight key differences between existing work and the researcher’s proposed approach.

Functional Requirements.

- The system must retrieve relevant articles based on the research direction provided by the user.
- The system must be capable of identifying and displaying key differences between the user’s research direction and existing work.
- If the system does not find relevant articles, it must not fabricate articles in order to provide an answer.
- The answers must be in the same language as the question. This means that responses should match the language in which the user asks the question, which should also correspond to the language of your dataset. As an exception, the titles of the articles may remain in their original language.

Below is an example of the expected interaction with the system:

Researcher:

I am researching implementations of federated neural topic models.

System:

I have found several relevant articles, including “Federated topic modeling” and “Federated non-negative matrix factorization for short texts topic modeling with mutual information”. These works focus on implementing federated approaches to Bayesian topic models, such as LDA or NMF, but none provide an implementation based on neural topic models.

Researcher:

I am researching systems for assisting researchers.

System:

I’m sorry, but I do not have any additional specific articles in the current corpus on this topic. This does not imply that none exist. You may try expanding your search in other databases.

Database. As a source of information for searching academic articles, we must rely on traditional databases, such as:

- Articles published in ACL. For example, this dataset⁸ contains 80 000 articles/posters dated up to September 2022.
- PubMed. A collection of scientific articles on topics related to medicine or health sciences, available through its website⁹.
- arXiv. Kaggle¹⁰ offers a dataset with more than 1.7 million abstracts and metadata of articles from arXiv.

Additional Features.

- The system can provide similarity metrics between the proposed idea and the articles found.
- Automatic state-of-the-art classification: The system can organize the retrieved articles into different categories based on their relevance, methodological approach, or contributions.
- For each retrieved article, the system can generate automatic summaries that highlight the key points. For this, a Transformer model fine-tuned on the training corpus and/or an LLM can be used.
- Allow the system to handle queries in multiple languages, translating both the questions and the answers if necessary, using pre-trained (or fine-tuned) translation models.

⁸<https://paperswithcode.com/dataset/acl-anthology-corpus-with-full-text>

⁹<https://ftp.ncbi.nlm.nih.gov/pubmed/baseline/>

¹⁰<https://www.kaggle.com/datasets/Cornell-University/arxiv>

6 Deployment Environment at UC3M

Students will have access to the LLMs deployed at UC3M. These models can be queried using a `curl` command, as shown in code block 1, or via the technologies covered in class. Note that for any of the above technologies, students must configure the LLM host to query the URL `yyuan.tsc.uc3m.es` using the API key provided in the `curl` example (1).

```
curl -k -H "X-API-KEY:sk-master-aaaac62102e9ec471ac2865db3dbb2fe"  
→ https://yyuan.tsc.uc3m.es/api/generate -d '{  
  "model": "llama3.1:8b",  
  "prompt": "Why is the sky blue?",  
  "stream": false  
}'
```

Figure 1: Example call to the llama3.1:8b model using curl.

The currently deployed LLMs are:

- llama3.1:8b
- qwen3:8b
- gemma3:4b

Students may also request the [deployment of other models](#) if needed for their assignment.